

PegShield: An On-Chain Risk Oracle Publishing Enforceable Collateral Constraints for Liquid-Staked Assets on Solana

Anubhav Prasai

AI Youth Global Organization (AIYGO)

(Dated: June 1, 2026)

Lending protocols govern liquid-staking-token (LST) collateral with static, governance-set loan-to-value (LTV) factors that are revised on the timescale of human risk review. During a de-pegging event, the borrow path moves faster than governance can respond, and the same composability that makes an LST capital efficient turns it into a correlated, illiquid collateral object. Price oracles do not close this gap: they report what an asset is worth, not whether a lender should keep extending credit against it. We present PegShield, a Solana-native *risk oracle* that publishes an enforceable lending constraint rather than a price. Off-chain processes estimate peg-deviation dynamics from Pyth market prices normalized by each token’s canonical staking exchange rate, fit an Ornstein–Uhlenbeck (OU) mean-reversion model, detect distress with a joint Augmented Dickey–Fuller (ADF) and z-score regime rule, and apply bounded liquidity and data-quality haircuts. The result is committed as a compact fixed-point risk state—a suggested LTV in basis points plus a regime flag—to a Solana program-derived address (PDA) that a lending market can read and enforce directly inside its borrow logic. We formalize the published constraint as a normalized solvency index $\Psi_t \in [\frac{1}{2}, 1]$, describe a bonded threshold-attester update path with disputes and slashing that preserves the consumer interface, and evaluate the system on a historical replay of the June 2022 stETH/ETH discount together with a battery of synthetic stress scenarios. In the replay, the model emits its first protective signal before a fixed-80% baseline exhibits modeled shortfall; we report the resulting loss reduction as an upper bound on the timing benefit rather than a causal guarantee. We are explicit about what this evidence does and does not establish, and about the system’s pre-production status. The contribution is a concrete primitive: peg-deviation tail risk converted into a machine-readable, on-chain credit boundary that downstream protocols can clamp, fall back from, or use to halt new credit.

I. INTRODUCTION

Liquid staking resolves a capital-efficiency problem in proof-of-stake systems: staked assets need not remain economically inert. On Solana, a high-throughput proof-of-stake blockchain [1], receipt tokens such as mSOL, jitoSOL, and bSOL represent claims on delegated SOL and circulate through automated market makers, lending markets, and vaults [2–4]. Liquid staking is now one of the largest categories of locked value in decentralized finance (DeFi) [5], and a substantial fraction of that value is pledged as collateral in on-chain lending markets.

This composability produces a structural tension we call the *liquidity-staking paradox*: the same token that improves capital efficiency under normal conditions becomes an unstable collateral object under stress. A stake-pool receipt is redeemable against an underlying staking position, but immediate secondary-market liquidity is finite and exit from the staking position is subject to unbonding and withdrawal friction. When the receipt trades below its canonical staking exchange rate, borrowers, liquidators, lenders, and arbitrageurs may all demand exit liquidity in the same short interval. The June 2022 stETH/ETH discount on Ethereum—during which a liquid-staking receipt traded persistently below parity with its underlying while leveraged holders deleveraged—is the canonical illustration of how a liquid-staking peg can widen faster than risk parameters are revised [6].

The prevailing control for this risk is the collateral factor (equivalently, the maximum LTV) set by protocol governance and periodically adjusted by risk contributors [7–9]. Governance-set parameters are deliberate and auditable, but they are revised on a human timescale—proposals, votes, timelocks—whereas a borrow transaction executes in a single slot. Empirical studies of DeFi liquidations document how quickly collateral quality can deteriorate and how liquidation activity clusters in stress windows [10–13]. The mismatch between governance latency and borrow-path latency is precisely the window in which new, undercollateralized credit is created.

Price oracles—Chainlink, Pyth, Uniswap time-weighted average prices (TWAPs), and the Maker oracle security module—supply the value of collateral to these markets [14–17]. They are necessary but not sufficient for the problem above: a price feed answers “what is this token worth?” and is itself a documented manipulation surface [18–21]. It does not answer the operational question a lender faces during a depeg: *given current peg deviation, estimated mean reversion, volatility, stationarity, liquidity depth, and source quality, should this market extend new credit against this collateral, and at what LTV?*

PegShield answers that question and commits the answer on-chain. It separates price discovery from collateral-solvency control: a price oracle continues to report value, while PegShield publishes a time-varying,

enforceable lending constraint keyed to a liquid-staking market. The answer lives in a Solana PDA, not an off-chain dashboard, so a lending program can read the account, verify ownership and freshness, and apply the suggested LTV (or a stricter local policy) inside its own borrow path.

a. Contributions. This paper makes the following contributions.

1. **A risk-oracle primitive for LST collateral.** We define an on-chain artifact whose output is an enforceable collateral constraint—a suggested LTV in basis points and a regime flag—rather than a price, and we formalize it as a normalized solvency index $\Psi_t \in [\frac{1}{2}, 1]$ (Section IV).
2. **A peg-deviation risk model.** We calibrate an Ornstein–Uhlenbeck process to a peg-deviation signal that normalizes the market LST/SOL ratio by the token’s canonical staking exchange rate, and we classify distress with a joint ADF/z-score regime rule, augmented by bounded liquidity and data-quality haircuts (Sections III–IV).
3. **An on-chain commitment and decentralized update path.** We describe a compact fixed-point risk-state account and a bonded threshold-attester cycle with disputes and slashing that decentralizes the writer while preserving a single, stable consumer interface (Sections III and V).
4. **A reference enforcement model and an evaluation.** We give a reference borrow gate that consumes the risk state, and we evaluate the system on a historical stETH/ETH replay and a suite of synthetic stress scenarios, stating precisely what the evidence supports (Sections VI–VII).

The remainder of the paper reviews related work (Section II), describes the architecture (Section III) and statistical model (Section IV), details the attester path (Section V) and consumer integration (Section VI), reports the evaluation (Section VII), analyzes security (Section VIII), and discusses limitations and threats to validity (Section IX).

II. BACKGROUND AND RELATED WORK

A. Blockchain Oracles and Their Manipulation

On-chain contracts cannot natively observe off-chain state, so oracles bridge external data onto the chain [22]. Chainlink aggregates reports from a decentralized operator set [14]; Pyth operates a first-party, pull-based model in which publishers sign prices that consumers pull on demand, each carrying a confidence interval [15, 23]; Uniswap v3 exposes a manipulation-resistant TWAP accumulator [16]; and MakerDAO interposes an Oracle Security Module that delays price updates to bound the

impact of a compromised feed [17]. A complementary design is the *optimistic* oracle—most prominently UMA—in which a proposed value is accepted unless disputed within a challenge window, with bonded escalation to a vote [24].

Oracles are also a primary attack surface. Eskandari et al. provide a systematization of oracle designs and the market-manipulation attacks they enable [18]; broader systematizations of DeFi attacks catalogue oracle-manipulation incidents as a dominant loss category [19]. Flash-loan-funded manipulation [20, 25] and the October 2022 Mango Markets exploit—in which an attacker inflated the price of thinly traded collateral to borrow against it [21]—demonstrate that feeding a manipulated price into a lending market’s collateral logic is sufficient to extract value; automated frameworks now exist to analyze such economic-security attacks at the smart-contract level [26]. PegShield’s threshold-attester path with disputes and slashing (Section V) is in the lineage of bonded and optimistic oracle security models [14, 24], but it publishes a risk constraint rather than a price, and it normalizes by a protocol reference rate to reduce sensitivity to ordinary USD-price drift.

B. Lending Protocols, Liquidations, and Systemic Risk

DeFi money markets such as Aave, Compound, and MakerDAO secure overcollateralized loans with per-asset collateral factors and liquidation thresholds set by governance [7–9]; Bartoletti et al. formalize the lending-pool abstraction these protocols share [27]. A substantial empirical and modeling literature studies what happens when collateral quality moves faster than these parameters. Qin et al. measure liquidation incentives, risks, and instabilities across major protocols [10]; Perez et al. show that lending sits “on a knife-edge,” with large positions liable to cascade [11]; Gudgeon et al. analyze the March 2020 “Black Thursday” deleveraging as a decentralized financial crisis [12, 28]; and Warmuz et al. characterize toxic liquidation spirals in which liquidations themselves degrade collateral value [13]. Klages-Mundt and Minca model deleveraging spirals and the stability of non-custodial pegged assets [29–31], which is directly relevant to LST peg dynamics. Broad systematizations of DeFi place these mechanisms in context [32]. PegShield targets exactly the window these works identify—the interval in which a static collateral factor is stale relative to deteriorating collateral—by making the collateral factor a live, on-chain function of measured peg risk.

C. Risk-Parameter Management and the Gap PegShield Fills

Specialized firms such as Gauntlet and Chaos Labs provide model-driven risk management to lending protocols,

using agent-based and Monte Carlo simulation to recommend collateral factors, supply caps, and liquidation parameters [33, 34]. These services are methodologically sophisticated, but their output is typically an *off-chain recommendation* that enters a protocol through governance, and is therefore subject to the same revision latency described above. Table I contrasts the available approaches along the axes that matter for our problem: what is published, whether it is directly enforceable on-chain in the borrow path, whether it is aware of LST peg dynamics specifically, and the latency of an update.

PegShield does not claim to replace any of these. It composes with a price oracle (it consumes Pyth) and leaves final authority with the lending protocol. Its distinct position is the combination of an *enforceable on-chain* output, a signal *specific to LST peg deviation*, and a *feed-speed* update path.

D. Statistical Foundations

The model rests on classical tools. The Ornstein–Uhlenbeck process [35] is the canonical continuous-time mean-reverting diffusion; it underlies the Vasicek term-structure model [36], Schwartz’s commodity-price model [37], and statistical-arbitrage and pairs-trading strategies that trade a mean-reverting spread [38, 39]. We estimate OU parameters from the discretized first-order autoregressive (AR(1)) regression of the increments, a standard approach [40]. Stationarity is assessed with the Augmented Dickey–Fuller unit-root test [41, 42]; the structural-break literature [43, 44] motivates treating a loss of stationarity, rather than a single large deviation, as the signature of a regime change. The peg-deviation series plays the role of the mean-reverting spread in pairs trading: healthy when stationary around zero, distressed when it drifts persistently negative.

III. SYSTEM ARCHITECTURE

A. Data Plane

Let a denote a liquid-staking market and s denote SOL. An off-chain bridge reads two Pyth market prices, P_t^a and P_t^s , and a protocol-specific reference exchange rate R_t^a . The reference rates are sourced per asset:

- the Marinade exchange rate for mSOL [2];
- Jito stake-pool statistics for jitoSOL [3];
- BlazeStake stake-pool account data read over a Solana remote procedure call (RPC) endpoint for bSOL [4].

The canonical risk signal is not the raw USD premium but the *peg deviation*

$$X_t^a = \frac{P_t^a/P_t^s}{R_t^a} - 1. \quad (1)$$

This distinction is essential. An LST trades above SOL in USD terms simply because its staking exchange rate accrues rewards; a naive USD spread therefore overstates risk and is dominated by yield accrual rather than peg stress. Peg risk begins when the market ratio P_t^a/P_t^s trades *below* the protocol redemption ratio R_t^a , i.e. when $X_t^a < 0$. The bridge rejects malformed price data, carries the Pyth confidence interval for each feed, records whether historical points are live or cache-derived, and marks reference-rate fallbacks. The engine treats degraded input quality as a distinct source of risk rather than a neutral missing value (Section IV). Figure 1 shows the end-to-end data plane.

B. Risk-State Commitment

The on-chain artifact is an Anchor account **RiskState** [45], a program-derived address [46] seeded by the market identifier:

$$\text{PDA}_a = \text{findProgramAddress}(["\text{risk}", \text{bytes}(a)], \text{program_id}). \quad (2)$$

The implemented fields are listed in Table II. Storing the state per market under a deterministic seed gives consumers a single, stable address to read regardless of how the write path is governed.

The oracle program performs no floating-point computation on-chain. Its role is to validate account identity, signer authority, update mode, numeric bounds, timing, and threshold-attester state transitions. In single-attester mode the update instruction requires the stored **authority** as signer, rejects accounts in multi-attester

mode, checks that the supplied identifier matches the PDA state, enforces $\theta_{\text{scaled}} \geq 0$, $\sigma_{\text{scaled}} > 0$, and $0 \leq \text{LTV}_{\text{bps}} \leq 10000$, and rate-limits updates to at least 30 seconds apart. This keeps the trusted on-chain logic small and auditable while the stochastic estimation remains off-chain.

TABLE I. Positioning of PegShield relative to existing on-chain data and risk tooling. “Enforceable on-chain” means the output can be read and applied by a lending program inside a borrow transaction without a governance action. PegShield’s “Seconds” update latency is the designed feed cadence / rate-limit floor (≥ 30 s, Section III), an architectural property of the pre-production system rather than a measured end-to-end latency.

System	Published output	Enforceable on-chain	on- LST peg aware	Update latency
Price oracle (Chain-link, Pyth, TWAP)	Asset price / confidence	Yes (price only)	No	Seconds–minutes
Governance factor (Aave, Compound, Maker)	Static max LTV / threshold	Yes	Indirect	Hours–weeks (governance)
Risk-modeling (Gauntlet, Labs)	service Off-chain parameter recommendation	No (via governance)	gover- Partial	Days (review cycle)
Optimistic (UMA)	oracle Arbitrary disputed value	Yes	No	Minutes–hours (challenge)
PegShield work)	(this Suggested LTV + regime flag	Yes (borrow path)	Yes (peg-normalized)	Seconds (feed update)

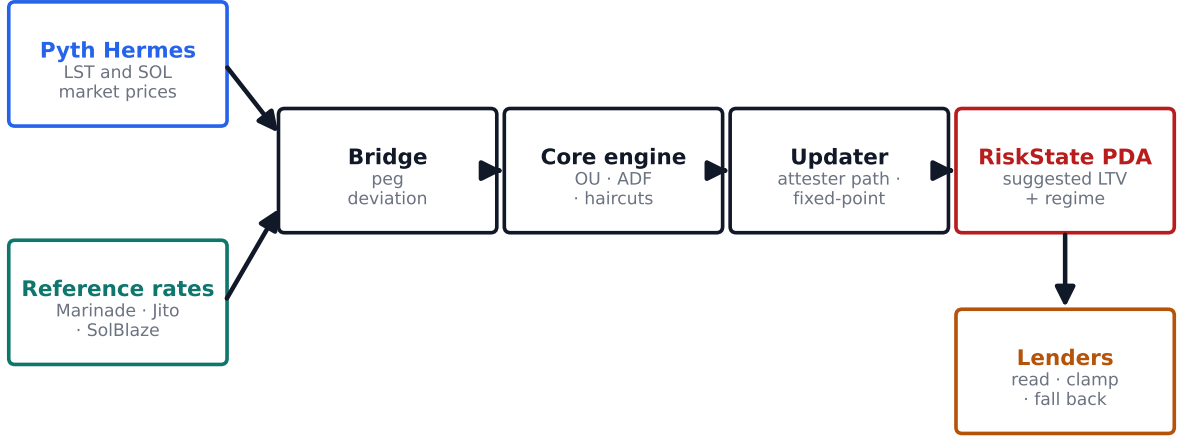


FIG. 1. PegShield data plane. Pyth market feeds and protocol reference rates are normalized by the bridge into the peg-deviation series of Eq. (1), calibrated by the off-chain engine, encoded into a fixed-point payload by the updater or attester path, and committed to a RiskState PDA consumed by lending protocols.

IV. STATISTICAL MODEL AND SOLVENCY INDEX

A. Ornstein–Uhlenbeck Calibration

The engine fits an OU model [35] to the peg-deviation series of Eq. (1). In continuous time,

$$dX_t = \theta(\mu - X_t) dt + \sigma dW_t, \quad (3)$$

where $\theta > 0$ is the mean-reversion speed, μ the local equilibrium deviation, $\sigma > 0$ the instantaneous volatility, and W_t a standard Brownian motion. We estimate the discretized AR(1) regression on the increments [36, 40],

$$\Delta X_i = \alpha + \beta X_{i-1} + \varepsilon_i, \quad (4)$$

with sampling interval Δt in days, and map the estimates back to OU parameters via

$$\hat{\theta} = -\frac{\hat{\beta}}{\Delta t}, \quad \hat{\mu} = \frac{\hat{\alpha}}{-\hat{\beta}}, \quad \hat{\sigma} = \frac{\text{sd}(\hat{\varepsilon})}{\sqrt{\Delta t}}. \quad (5)$$

Positive lower bounds ($\theta, \sigma \geq 10^{-6}$) prevent degenerate zero estimates. The baseline parameters $(\hat{\theta}_t, \hat{\sigma}_t)$ are medians of $\hat{\theta}$ and $\hat{\sigma}$ over rolling windows when sufficient history exists, and the current-window estimate otherwise. Intuitively, a healthy peg reverts quickly with low volatility (large θ , small σ); a stressed peg reverts slowly or not at all (small θ) with elevated σ .

TABLE II. RiskState account fields. Real-valued parameters are stored in fixed point at scale 10^6 .

Field	Type	Meaning
lst_id	String	Market identifier, max 16 bytes.
theta_scaled	i64	OU mean-reversion speed $\theta \times 10^6$.
sigma_scaled	i64	OU volatility $\sigma \times 10^6$.
z_score_scaled	i64	Current z-score $z \times 10^6$.
regime_flag	u8	0 = normal, 1 = critical.
suggested_ltv_bps	u16	Suggested LTV in basis points.
slot	u64	Slot of last accepted update.
timestamp	i64	Unix time of last accepted update.
authority	Pubkey	Writer in single-attester mode.
last_updater	Pubkey	Most recent submitting signer.
update_mode	u8	0 = single-attester, 1 = multi-attester.
attester_registry	Pubkey	Registry PDA in multi-attester mode.

B. Regime Detection

The current standardized deviation over the rolling window $[t - n, t]$ is

$$z_t = \frac{X_t - \bar{X}_{[t-n, t]}}{s_{[t-n, t]}}. \quad (6)$$

We additionally apply the Augmented Dickey–Fuller test [41, 42] to the rolling window, yielding a unit-root p -value $p_{\text{ADF}, t}$. The critical-regime indicator is the *conjunction*

$$C_t = \mathbf{1}\{|z_t| > 2.5\} \cdot \mathbf{1}\{p_{\text{ADF}, t} \geq 0.05\}. \quad (7)$$

That is, **CRITICAL** requires both an extreme standardized deviation *and* failure to reject the unit root (loss of stationarity). Requiring both conditions avoids treating an

ordinary large variance realization in an otherwise mean-reverting series as a structural break, consistent with the distinction between a transient deviation and a regime change drawn in the structural-break literature [43, 44].

C. Liquidity and Data-Quality Risk

LST solvency depends on more than the price path. Let L_t denote effective exit liquidity available to liquidators and arbitrageurs and let $H_t \in [0, 1]$ denote a latent stake-pool health variable capturing delegation performance, withdrawal friction, and reference-rate reliability. Peg stability can be modeled as a jump-diffusion extension of Eq. (3),

$$dX_t = \theta_t(\mu_t - X_t)dt + \sigma_t dW_t - \kappa_L dN_t^L - \kappa_H dN_t^H, \quad (8)$$

where liquidity and health shocks arrive with state-dependent intensities

$$\lambda_t^L = \lambda_0^L \exp\left(\beta_L \left(1 - \frac{L_t}{L_t^*}\right)_+\right), \quad \lambda_t^H = \lambda_0^H \exp\left(\beta_H (1 - H_t)_+\right). \quad (9)$$

Equation (8) is the model that motivates treating liquidity and source degradation as first-class risk; the present implementation does not yet ingest validator-performance telemetry as a live field. It uses the canonical staking exchange rate as the accounting reference and converts available signals into two bounded haircuts: a liquidity haircut h_t^L computed from generic depth, slippage, pool-imbalance, withdrawal-delay, and concentration metrics when present, and a data-quality haircut h_t^Q computed from Pyth confidence ratios, history-source fallback, reference-rate fallback, and the absence of liquidity metrics. Both are bounded:

$$h_t^L \in [0, 0.30], \quad h_t^Q \in [0, 0.15]. \quad (10)$$

D. The Published LTV and the Solvency Index

With base constants $\ell_0 = 0.80$, $\ell_{\min} = 0.40$, and $\ell_{\max} = 0.80$, the statistical LTV is

$$\ell_t^{\text{stat}} = \begin{cases} \ell_{\min}, & C_t = 1, \\ \text{clip}\left(\ell_0 \frac{\hat{\theta}_t}{\bar{\theta}_t} \frac{\bar{\sigma}_t}{\hat{\sigma}_t}, \ell_{\min}, \ell_{\max}\right), & C_t = 0. \end{cases} \quad (11)$$

The multiplicative form rewards fast reversion and penalizes elevated volatility relative to the rolling baseline; a critical regime collapses the LTV to the floor unconditionally. The published LTV applies the haircuts and

re-clips:

$$\ell_t = \text{clip}\left(\ell_t^{\text{stat}} - h_t^L - h_t^Q, \ell_{\min}, \ell_{\max}\right). \quad (12)$$

We summarize the enforced constraint by the normalized *solvency index*

$$\Psi_t = \frac{\ell_t}{\ell_0} \in \left[\frac{1}{2}, 1\right], \quad (13)$$

where $\Psi_t = 1$ permits the base collateral factor and $\Psi_t = \frac{1}{2}$ is the configured 40% floor. For a lending market with collateral value V_t and requested debt B_t , PegShield’s reference credit constraint is

$$B_t \leq V_t \ell_t. \quad (14)$$

The protocol remains free to impose a stricter cap, fall back, pause, or apply its own liquidation policy; PegShield supplies the time-varying upper bound, not the final decision.

E. Off-Chain to On-Chain Serialization

The boundary between stochastic off-chain estimation and deterministic on-chain state is a fixed-point encoding at scale 10^6 , with the suggested LTV converted to basis points:

```
theta_scaled = round(theta * 1_000_000)
sigma_scaled = round(sigma * 1_000_000)
z_score_scaled = round(z_score * 1_000_000)
suggested_ltv_bps = round(clip(ell, 0, 1) * 10_000)
```

The on-chain program validates the bounds in Section III but never recomputes the model; the integrity of the estimation is enforced socially and economically through the attester path rather than re-executed in the runtime.

V. DECENTRALIZED UPDATES: THRESHOLD ATTESTERS

A single trusted writer is a single point of failure. The implemented multi-attester path decentralizes the writer while preserving the consumer interface: protocols still read the same **RiskState** PDA, and the change is confined to how writes are authorized. The design borrows from bonded and optimistic oracle security models [14, 24]: attesters post a bond, propose and confirm updates by threshold, and can be disputed and slashed for bad updates within a challenge window. Table III lists the implemented constants and accounts.

The update cycle, illustrated in Fig. 2, is:

1. **initialize_registry(threshold, min_bond)** creates the registry PDA and sets admin, threshold, minimum bond, and slash destination.

TABLE III. Implemented threshold-attester constants and accounts.

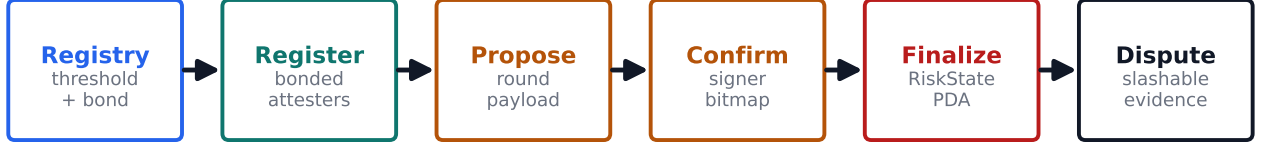
Item	Implemented behavior
MAX_ATTESTERS	5 fixed registry slots.
MIN_ATTESTER_BOND	1 SOL minimum registration bond.
PENDING_UPDATE_TTL	300 seconds.
DISPUTE_WINDOW	3600 seconds after finalization.
RESOLUTION_DEADLINE	24 hours after a dispute is filed.
UNREGISTER_COOLDOWN	7 days before bond withdrawal.
SLASH_PERCENT	50% of the attester bond.
DISPUTER_REWARD	50% of the slashed amount.
AttesterRegistry	Admin, threshold, minimum bond, slash destination, attester entries.
PendingUpdate	Round payload, proposer, expiry, confirmation bitmap, finalization metadata.
DisputeRecord	Evidence hash, disputed attester, deadline, outcome, copied parameters.

2. **register_attester(bond)** transfers lamports from the attester to the registry PDA and records an active slot; bond withdrawal is subject to a 7-day cooldown.
3. **enable_multi_attester(1st_id)** switches a **RiskState** from single- to multi-attester mode once the registry can satisfy its threshold.
4. **propose_update** opens a pending round; the proposer must be active and auto-confirms via a bitmap bit indexed by registry position.
5. **confirm_update** requires a registered, not-yet-confirming attester and a non-expired pending round. When the confirmation count reaches the threshold, the risk parameters are copied into **RiskState** and the round is finalized.
6. **cancel_expired** closes an expired unfinalized round and refunds rent to the proposer.
7. **dispute_update** may be filed within the dispute window against an attester that participated in the round.
8. **resolve_dispute** is admin-gated; on a slash, 50% of the attester’s bond is deducted, half to the disputer and half to the configured destination.

The program authenticates attesters by Solana transaction-signer identity and registry membership. It does not yet implement an on-chain Ed25519 aggregate-signature verifier over an off-chain canonical payload; the readiness of the state machine is therefore distinct from the operation of an independent production committee, a gap we return to in Section IX.

VI. CONSUMER INTEGRATION

PegShield is designed to be consumed in three ways: off-chain or client-side through a typed client library, directly inside another Anchor program by deriving and decoding the PDA, and through a reference borrow gate that records auditable decisions. The first two are mechanical; we describe the reference policy and the borrow gate, which encode the intended asymmetric use of the feed.



Consumers continue to read the same RiskState PDA; only the write trust model changes.

FIG. 2. Threshold-attester lifecycle. The consumer PDA remains stable while the write path moves from a single authority to bonded proposal, threshold confirmation, finalization, and dispute resolution.

A. Reference Consumer Policy

A conservative consumer treats staleness and critical regime as reasons to *reduce* exposure, never to increase

it. The reference policy maps the on-chain state S_t to an applied LTV by

$$\text{safeLtv}(S_t) = \begin{cases} \ell_{\text{fallback}}, & \text{stale}(S_t), \\ \ell_{\text{fallback}}, & S_t.\text{regimeFlag} = 1, \\ \min(\max(S_t.\text{suggestedLtv}, 0), \ell_{\text{protocolMax}}), & \text{otherwise,} \end{cases} \quad (15)$$

where staleness defaults to an age beyond 600 seconds and an account whose timestamp is zero is treated as stale. A client verifies that the account is owned by the expected program and decodes the fixed-point layout before applying Eq. (15); the default fallback is 40% and example consumers cap the maximum at a protocol-selected ceiling.

account constraint, validates that the identifiers match, and records a borrow decision with the applied LTV, oracle age, a reason code, and an allow/reject outcome. Its decision logic is

B. On-Chain Borrow Gate

The reference borrow gate is an Anchor program that reads the PegShield RiskState through a cross-program

$$\ell_t^{\text{applied}} = \begin{cases} 0, & \text{policy paused,} \\ \ell_{\text{fallback}}, & \text{oracle stale,} \\ 0, & \text{critical and halt-on-critical,} \\ \ell_{\text{fallback}}, & \text{critical and not halt-on-critical,} \\ \min(\ell_t^{\text{oracle}}, \ell_{\text{policyMax}}), & \text{otherwise,} \end{cases} \quad (16)$$

and the borrow is permitted only when the requested

debt does not exceed $V_t \ell_t^{\text{applied}}$. The gate demonstrates

that the published constraint is enforceable inside a borrow transaction without any governance action, which is the property that distinguishes PegShield from an off-chain parameter recommendation (Table I).

VII. EVALUATION

A. Methodology

We evaluate the policy surface in two complementary ways. First, a *historical replay* drives the engine with reconstructed market data from the June 2022 stETH/ETH discount and compares the resulting dynamic LTV against a fixed 80% collateral factor on a modeled collateral position. Second, a *scenario suite* drives the engine with synthetic stress paths to exercise the policy under shapes of stress that are rare in the short on-chain history of Solana LSTs. We report what each method establishes and, equally, what it does not.

The historical replay uses archived ETH daily closes and published stETH discount anchors interpolated into a daily window from 18 May to 30 June 2022, with a warm-up of 20 points preceding 24 replay rows. The collateral model is 100 units of a stETH-like asset; “shortfall” is the scenario-scale dollar amount by which a position’s debt exceeds its collateral value under each policy. The dynamic borrower is modeled as listening to the most conservative LTV PegShield reaches during the event, which isolates the value of *timing*—tightening before the worst drawdown—rather than modeling liquidator microstructure.

B. Historical stETH/ETH Replay

Table IV and Figs. 3–4 summarize the replay. PegShield emits its first critical signal on 2022-06-09 at a peg deviation of -3.40% , four days before the fixed-80% baseline first exhibits modeled shortfall on 2022-06-13. Over the window, the fixed-80% policy reaches a peak modeled shortfall of \$51,946.38 on the 100-unit position, while the PegShield policy—already at its 40% floor—records \$0.00 modeled shortfall, a peak LTV cut of 40 percentage points (Ψ_t falling from 1 to $\frac{1}{2}$).

The takeaway is timing: a feed-speed risk constraint emits its first protective signal (the CRITICAL flag on 2022-06-09) before a static collateral factor produces visible shortfall in the replay. We treat the \$0.00 PegShield shortfall as an *upper bound* on the achievable timing benefit, not a causal guarantee, and we discuss the look-ahead nature of the modeled borrower in Section IX.

C. Synthetic Stress Suite

To check the policy beyond a single event, we drive the engine with eight synthetic stress paths spanning comple-

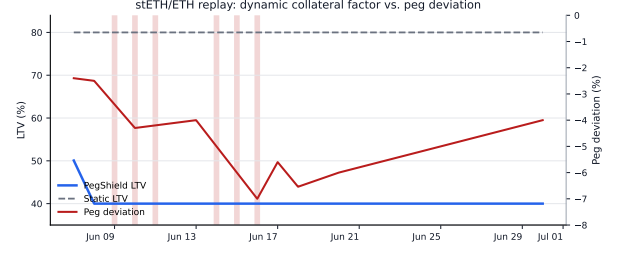


FIG. 3. Historical stETH/ETH replay. The dynamic LTV falls from the static 80% baseline to the 40% floor as peg deviation deteriorates, i.e. Ψ_t moves from 1 to $\frac{1}{2}$.

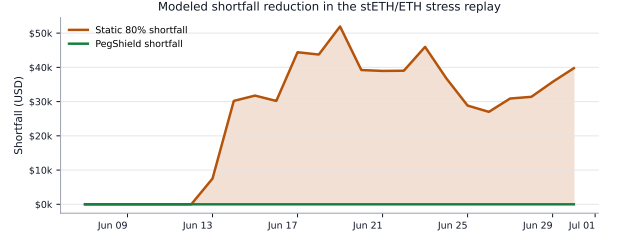


FIG. 4. Modeled shortfall for the same replay. The static 80% policy develops positive shortfall as the discount widens; the PegShield policy has already tightened to the floor.

mentary failure modes, and we report the same shortfall and regime metrics as for the historical replay (Table V). The paths are constructed on the modeled 100-unit position used throughout, and the metrics are computed by the same engine and policy code that produces the on-chain state.

Two properties are worth separating. First, *loss prevention*: in three of the nine scenarios the fixed-80% baseline incurs modeled shortfall while PegShield does not, for an aggregate modeled loss prevented of \$52,143 on the idealized position—a figure dominated by the historical replay, with the two synthetic deleveraging paths contributing small amounts. Second, and more informative for the synthetic suite, is *regime behavior*: in the remaining six scenarios neither policy breaches the idealized 80% position, yet PegShield still tightens during the stress window (reflected in the critical-row counts) and then recovers to monitoring rather than latching at the floor. The `false_positive_wick` path is the control: a single large but stationary deviation produces only four critical rows and a prompt recovery, because the conjunction in Eq. (7) requires loss of stationarity, not merely a large z . Sustained, non-stationary deterioration—the `stablecoin_redemption_spiral` and `restaking_bridge_contagion` paths—instead produces long critical runs (22 and 20 rows). This is the intended precision/recall tradeoff: the detector reacts to structural breaks while tolerating transient noise. We caution that the synthetic shortfall figures are small precisely because the idealized 100-unit position is conservative; they characterize the *policy surface*, not

TABLE IV. Historical stETH/ETH replay (selected rows). Shortfall is scenario-scale dollars on a modeled 100-unit position. The *PegShield LTV* column reports the running minimum (most conservative LTV reached to date) that the modeled borrower follows, which is why it remains at the 40% floor on **NORMAL** rows after a prior **CRITICAL** signal; consequently the PegShield shortfall is an *upper bound* on the timing benefit rather than a per-slot causal measurement (see Section IX).

Date	Peg dev.	z	Regime	PegShield LTV [†]	Static LTV	PS shortfall	Static shortfall
2022-06-07	-2.40%	-1.62	NORMAL	50.1%	80.0%	\$0.00	\$0.00
2022-06-09	-3.40%	-2.96	CRITICAL	40.0%	80.0%	\$0.00	\$0.00
2022-06-13	-4.00%	-1.88	NORMAL	40.0%	80.0%	\$0.00	\$7,525.44
2022-06-16	-7.00%	-2.97	CRITICAL	40.0%	80.0%	\$0.00	\$30,182.25
2022-06-19	-6.27%	-1.91	NORMAL	40.0%	80.0%	\$0.00	\$51,946.38
2022-06-30	-4.00%	-0.19	NORMAL	40.0%	80.0%	\$0.00	\$39,763.20

[†]Running minimum adopted by the modeled borrower, not the instantaneous per-slot LTV; the per-slot policy returns to a higher LTV on **NORMAL** rows.

TABLE V. Stress-suite results (one historical replay and eight synthetic paths) on the modeled 100-unit position. “Static shortfall” and “PegShield shortfall” are peak modeled dollars under the fixed-80% baseline and under PegShield; “Critical rows” counts rows flagged **CRITICAL** by Eq. (7); “Recovered” indicates the regime returned to monitoring rather than remaining latched at the floor. Synthetic paths are regression tests for the policy surface, not empirical market measurements.

Scenario	Risk focus	Static shortfall	PegShield shortfall	Critical rows	Recovered
steth_june_2022	Historical collateral impairment	\$51,946	\$0	6	Yes
reflexive_bank_run	Reflexive deleveraging	\$165	\$0	15	Yes
liquidity_vacuum	Liquidity vacuum	\$32	\$0	17	Yes
slow_grind_depeg	Slow creeping impairment	\$0	\$0	13	Yes
flash_crash_repricing	Flash crash / snapback	\$0	\$0	11	Yes
svb_weekend_bank_run	Weekend redemption panic	\$0	\$0	16	Yes
stablecoin_redemption_spiral	Reflexive liquidation spiral	\$0	\$0	22	Yes
restaking_bridge_contagion	Cross-venue contagion	\$0	\$0	20	Yes
false_positive_wick	Noise resilience	\$0	\$0	4	Yes

realized losses, liquidator behavior, or venue-specific liquidity.

VIII. SECURITY AND MANIPULATION ANALYSIS

A. Manipulation Surface

By normalizing the market LST/SOL ratio with a protocol reference rate (Eq. (1)), PegShield is less sensitive to ordinary USD-price drift than a naive LST/USD spread. To move the signal, an adversary must influence the relative LST/SOL market price, the Pyth feed, or the canonical reference-rate source. The implemented defenses are:

- on-chain writers cannot submit $\theta < 0$, $\sigma \leq 0$, or an LTV outside $[0, 10000]$ basis points;
- single-attester writes require the stored authority and are rate-limited to one update per 30 seconds;
- multi-attester writes require active bonded registry members and threshold confirmation before finalization, with a dispute-and-slash window;
- source degradation is represented as a data-quality haircut rather than silently ignored, and missing liquidity metrics are penalized;

- consumers verify account ownership, decode the layout, and check freshness and regime before use, with an asymmetric policy that lowers LTV or halts borrowing on stale, missing, or critical data.

These mechanisms reduce, but do not eliminate, manipulation risk. PegShield inherits a dependence on Pyth honesty and timeliness [15, 18] and on reference-rate integrity; it does not currently cross-check Pyth against a secondary oracle, and it does not recompute the OU/ADF logic on-chain. The Mango Markets incident [21] is a reminder that a manipulated input to collateral logic is sufficient for loss; the peg normalization and the asymmetric consumer policy are mitigations, not guarantees.

B. Composability

Because the primary output is a single PDA per market rather than a bespoke integration, a protocol can consume PegShield off-chain through a client, directly in an Anchor program by deriving `["risk", lst_id]` under the program ID, or through the reference borrow-gate pattern. The design deliberately does not require a lender to surrender final risk authority: protocols retain their own maximum LTVs, fallbacks, pauses, and critical-regime behavior, and PegShield supplies the time-varying

upper bound of Eq. (14).

IX. DISCUSSION, LIMITATIONS, AND THREATS TO VALIDITY

a. Deployment status. The system is pre-production. The program is deployed only to Solana devnet, the code is unaudited and upgrade-mutable, and an independent production attester committee is not yet operating. These are deliberate limitations of a research prototype, not claims about a live mainnet service.

b. Threats to the evaluation’s validity. The historical replay is a reconstruction, not a record of PegShield operating in 2022: ETH closes are archived and stETH discount points are published anchors interpolated into the window, so the exact daily path is approximate, and we have not tested whether the timing lead survives an alternative but equally defensible reconstruction. The shortfall figures are scenario-scale dollars on a single modeled 100-unit position, not market-wide losses. Most importantly, the modeled borrower is *non-causal*: it follows the running minimum of the LTV reached during the event rather than the per-slot LTV the policy would have published online. This is visible in Table IV, where the *PegShield LTV* column stays at the floor on **NORMAL** rows that follow a prior **CRITICAL** signal, whereas the per-slot functional (Eq. (11)) only forces the floor when $C_t = 1$. The \$0.00 PegShield shortfall is therefore an *upper bound* on the timing benefit, not a measured causal property; a causal online policy is the correct primary metric and is left to future work. The replay also does not model liquidation auction latency, keeper competition, gas, venue-specific liquidity, or borrower behavior after a margin call—all factors the liquidation literature shows are decisive in practice [10, 11, 13]. The synthetic scenarios are regression tests, not empirical measurements. We therefore claim that the model *signals* earlier than a static baseline in the replay, and we do *not* claim zero market-wide bad debt, production-grade liquidator performance, or calibration validity across all LSTs.

c. Statistical-method limitations. Two properties of the estimator warrant caution. First, the Augmented Dickey–Fuller test in Eq. (7) runs on rolling windows of roughly twenty observations, a regime in which the unit-root test has low power and unreliable size; the “failure to reject stationarity” half of the critical conjunction is thus statistically weak at this window length, and in practice the z-score component carries most of the detection. A full power analysis and a sensitivity study over window length and ADF lag selection are needed before the joint rule can be claimed to outperform a z-score-only detector. Second, the OU mapping in Eq. (5) is well defined only for a mean-reverting fit ($\hat{\beta} < 0$); when a window is non-stationary or trending ($\hat{\beta} \geq 0$), $\hat{\theta}$ is clamped at its positive floor and $\hat{\mu}$ becomes ill conditioned. This is precisely the distressed regime the system targets, so we treat a non-mean-reverting fit as itself a distress indicator

rather than a reliable parameter estimate; making that fallback explicit and analyzing it is future work.

d. Construct and external validity. The LTV functional (Eq. (11)) and the regime thresholds (Eq. (7)) embed constants—the 0.80 base factor, the 0.40 floor, the 2.5 z-score threshold, the 0.05 ADF level, and the haircut bounds—that were chosen for the prototype and require per-asset, per-venue calibration before mainnet use. Solana LSTs also have a short on-chain history relative to the stress shapes of interest, which is why the historical anchor is an Ethereum event; whether Solana LST pegs exhibit the same dynamics is an empirical question this work does not settle.

e. Research agenda. The natural extensions follow from these limitations: a secondary price-source cross-check; ingestion of validator and stake-pool health telemetry as a live field feeding H_t in Eq. (8); an on-chain aggregate-signature verifier so that finalization attests to an off-chain canonical payload; formal verification of the LTV clamp and timing invariants; immutable or multisig-controlled upgrade authority; venue-specific liquidity-depth calibration per LST; and a forward-looking, mainnet evaluation against realized borrow and liquidation outcomes.

X. CONCLUSION

We presented PegShield, a Solana-native risk oracle that publishes an enforceable collateral constraint for liquid-staking-token collateral rather than a price. PegShield estimates peg-deviation dynamics with an Ornstein–Uhlenbeck model, detects distress with a joint ADF/z-score regime rule, applies bounded liquidity and data-quality haircuts, and commits a compact fixed-point risk state—a suggested LTV and a regime flag—to a Solana PDA that a lending market can enforce directly through a bonded, disputable attester path. We formalized the published constraint as a normalized solvency index $\Psi_t \in [\frac{1}{2}, 1]$ and showed, in a historical stETH/ETH replay, that a feed-speed constraint signals before a static collateral factor exhibits modeled shortfall—reporting the loss reduction as an upper bound on the timing benefit—while being explicit about the prototype’s pre-production status and the limits of the evidence. The contribution is a concrete primitive that converts peg-deviation tail risk into a machine-readable, on-chain credit boundary, reframing LST collateral management as a continuous risk service rather than a static governance parameter.

a. Availability. The implementation—off-chain bridge and engine, the Anchor programs for the risk state, attester registry, and reference borrow gate, a typed client library, an operator interface, and the historical and synthetic stress harness—together with the data and scripts to reproduce the evaluation in Section VII, is available as an open-source repository at <https://github.com/Fianko-codes/PegShield>. For repro-

ducibility, a reference deployment of the oracle program exists on the Solana *devnet* under program identifier `DMR3rXBh8RGrKyx1mxqFVTMbyfoiuu9iYHr6s6CW23ea`,

with per-asset `RiskState` accounts derived as in Section III; this deployment is for demonstration and verification only and is not a production service.

-
- [1] A. Yakovenko, Solana: A new architecture for a high performance blockchain, Solana Labs Whitepaper (2018), <https://solana.com/solana-whitepaper.pdf>. Accessed 2026-05-31.
 - [2] Marinade Finance, What is mSOL?, <https://docs.marinade.finance/> (2026), accessed 2026-05-31.
 - [3] Jito Foundation, JitoSOL liquid staking, <https://www.jito.network/docs/jitosol/jitosol-liquid-staking/> (2026), accessed 2026-05-31.
 - [4] SolBlaze, BlazeStake liquid staking (bSOL), <https://stake-docs.solblaze.org/> (2026), accessed 2026-05-31.
 - [5] DeFiLlama, Liquid staking total value locked, <https://defillama.com/lsd> (2026), accessed 2026-05-31.
 - [6] Lido Finance, The stETH/ETH secondary-market discount of june 2022, <https://blog.lido.fi/> (2022), context on the June 2022 stETH discount during the Celsius/3AC deleveraging. Accessed 2026-05-31.
 - [7] Aave Labs, Aave protocol documentation: Liquidations, loan-to-value and health factor, <https://aave.com/docs> (2026), accessed 2026-05-31.
 - [8] R. Leshner and G. Hayes, Compound: The money market protocol, <https://compound.finance/documents/Compound.Whitepaper.pdf> (2019), accessed 2026-05-31.
 - [9] Maker Team, The Maker protocol: MakerDAO's multi-collateral Dai (MCD) system, <https://makerdao.com/whitepaper/> (2020), accessed 2026-05-31.
 - [10] K. Qin, L. Zhou, P. Gamito, P. Jovanovic, and A. Gervais, An empirical study of DeFi liquidations: Incentives, risks, and instabilities, in *Proceedings of the 21st ACM Internet Measurement Conference (IMC '21)* (ACM, 2021) pp. 336–350.
 - [11] D. Perez, S. M. Werner, J. Xu, and B. Livshits, Liquidations: DeFi on a knife-edge, in *Financial Cryptography and Data Security (FC 2021)*, Lecture Notes in Computer Science, Vol. 12675 (Springer, 2021) pp. 457–476.
 - [12] L. Gudgeon, D. Perez, D. Harz, B. Livshits, and A. Gervais, The decentralized financial crisis, in *2020 Crypto Valley Conference on Blockchain Technology (CVCBT)* (IEEE, 2020) pp. 1–15.
 - [13] J. Warmuz, A. Chaudhary, and D. Pinna, *Toxic liquidation spirals*, arXiv preprint arXiv:2212.07306 (2022).
 - [14] L. Breidenbach, C. Cachin, B. Chan, A. Coventry, S. Ellis, A. Juels, F. Koushanfar, A. Miller, B. Magauran, D. Moroz, S. Nazarov, A. Topliceanu, F. Tramèr, and F. Zhang, *Chainlink 2.0: Next Steps in the Evolution of Decentralized Oracle Networks*, Whitepaper (Chainlink Labs, 2021) <https://research.chain.link/whitepaper-v2.pdf>.
 - [15] Pyth Data Association, *Pyth Network: A First-Party Financial Oracle*, Whitepaper (Pyth Data Association, 2022) <https://pyth.network/whitepaper.pdf>.
 - [16] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, and D. Robinson, *Uniswap v3 Core*, Whitepaper (Uniswap Labs, 2021) <https://uniswap.org/whitepaper-v3.pdf>.
 - [17] MakerDAO, Oracle module: Oracle security module (OSM) and medianizer, <https://docs.makerdao.com/smart-contract-modules/oracle-module> (2023), accessed 2026-05-31.
 - [18] S. Eskandari, M. Salehi, W. C. Gu, and J. Clark, SoK: Oracles from the ground truth to market manipulation, in *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT '21)* (ACM, 2021) pp. 127–141.
 - [19] L. Zhou, X. Xiong, J. Ernstberger, S. Chaliasos, Z. Wang, Y. Wang, K. Qin, R. Wattenhofer, D. Song, and A. Gervais, SoK: Decentralized finance (DeFi) attacks, in *2023 IEEE Symposium on Security and Privacy (SP)* (IEEE, 2023) pp. 2444–2461.
 - [20] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels, Flash boys 2.0: Frontrunning, transaction reordering, and consensus instability in decentralized exchanges, in *2020 IEEE Symposium on Security and Privacy (SP)* (IEEE, 2020) pp. 910–927.
 - [21] rekt.news, Mango markets — REKT, <https://rekt.news/mango-markets-rekt/> (2022), post-mortem of the October 2022 Mango Markets oracle-price-manipulation exploit. Accessed 2026-05-31.
 - [22] A. Benîiche, A study of blockchain oracles, arXiv preprint arXiv:2004.07140 (2020), <https://arxiv.org/abs/2004.07140>.
 - [23] Pyth Network, Hermes: Pyth price-feed API, <https://docs.pyth.network/price-feeds/api-instances-and-providers/hermes> (2026), accessed 2026-05-31.
 - [24] UMA Protocol, How does UMA's optimistic oracle work?, <https://docs.uma.xyz/protocol-overview/how-does-umas-oracle-work> (2024), accessed 2026-05-31.
 - [25] K. Qin, L. Zhou, B. Livshits, and A. Gervais, Attacking the DeFi ecosystem with flash loans for fun and profit, in *Financial Cryptography and Data Security (FC 2021)*, Lecture Notes in Computer Science, Vol. 12674 (Springer, 2021) pp. 3–32.
 - [26] K. Babel, P. Daian, M. Kelkar, and A. Juels, Clockwork finance: Automated analysis of economic security in smart contracts, in *2023 IEEE Symposium on Security and Privacy (SP)* (IEEE, 2023) pp. 622–639.
 - [27] M. Bartoletti, J. H.-y. Chiang, and A. Lluch-Lafuente, SoK: Lending pools in decentralized finance, arXiv preprint arXiv:2012.13230 (2021).
 - [28] L. Gudgeon, S. M. Werner, D. Perez, and W. J. Knottenbelt, *DeFi protocols for loanable funds: Interest rates, liquidity and market efficiency*, arXiv preprint arXiv:2006.13922 (2020).
 - [29] A. Klages-Mundt and A. Minca, (In)Stability for the blockchain: Deleveraging spirals and stablecoin attacks, in *Cryptoeconomic Systems* (2021) arXiv:1906.02152.
 - [30] A. Klages-Mundt and A. Minca, While stability lasts: A stochastic model of non-custodial stablecoins, *Mathematical Finance* **32**, 943 (2022).
 - [31] A. Klages-Mundt, D. Harz, L. Gudgeon, J.-Y. Liu, and A. Minca, Stablecoins 2.0: Economic foundations and

- risk-based models, in *Proceedings of the 2nd ACM Conference on Advances in Financial Technologies (AFT '20)* (ACM, 2020) pp. 59–79.
- [32] S. M. Werner, D. Perez, L. Gudgeon, A. Klages-Mundt, D. Harz, and W. J. Knottenbelt, SoK: Decentralized finance (DeFi), in *Proceedings of the 4th ACM Conference on Advances in Financial Technologies (AFT '22)* (ACM, 2022) pp. 30–46.
- [33] Gauntlet, Gauntlet: Model-driven risk management for DeFi protocols, <https://www.gauntlet.xyz> (2026), accessed 2026-05-31.
- [34] Chaos Labs, Chaos labs: Agent-based and Monte Carlo risk simulations for DeFi protocols, <https://chaoslabs.xyz> (2026), accessed 2026-05-31.
- [35] G. E. Uhlenbeck and L. S. Ornstein, On the theory of the Brownian motion, *Physical Review* **36**, 823 (1930).
- [36] O. Vasicek, An equilibrium characterization of the term structure, *Journal of Financial Economics* **5**, 177 (1977).
- [37] E. S. Schwartz, The stochastic behavior of commodity prices: Implications for valuation and hedging, *The Journal of Finance* **52**, 923 (1997).
- [38] M. Avellaneda and J.-H. Lee, Statistical arbitrage in the US equities market, *Quantitative Finance* **10**, 761 (2010).
- [39] E. Gatev, W. N. Goetzmann, and K. G. Rouwenhorst, Pairs trading: Performance of a relative-value arbitrage rule, *The Review of Financial Studies* **19**, 797 (2006).
- [40] J. D. Hamilton, *Time Series Analysis* (Princeton University Press, Princeton, NJ, 1994).
- [41] D. A. Dickey and W. A. Fuller, Distribution of the estimators for autoregressive time series with a unit root, *Journal of the American Statistical Association* **74**, 427 (1979).
- [42] S. E. Said and D. A. Dickey, Testing for unit roots in autoregressive-moving average models of unknown order, *Biometrika* **71**, 599 (1984).
- [43] D. W. K. Andrews, Tests for parameter instability and structural change with unknown change point, *Econometrica* **61**, 821 (1993).
- [44] J. Bai and P. Perron, Computation and analysis of multiple structural change models, *Journal of Applied Econometrics* **18**, 1 (2003).
- [45] Anchor Maintainers, Anchor: A framework for solana's Sealevel runtime, <https://www.anchor-lang.com/docs> (2026), accessed 2026-05-31.
- [46] Solana Foundation, Program derived addresses (PDAs), <https://solana.com/docs/core/pda> (2026), accessed 2026-05-31.